



chat preciso da sua ajuda com um relatório sobre Go, pode me ajudar?

Claro. Pode mandar o **PDF, foto ou enunciado do relatório de Go** que eu te ajudo com tudo.

Posso fazer no padrão que você costuma usar para esses relatórios:

- códigos **bem simples, nível iniciante**
- **sem comentários**
- lógica mais fácil possível
- `print` curto, em minúsculo e sem acentos
- exercícios separados
- explicação só quando você pedir
- se precisar, também monto as **perguntas para o IA Report** depois

Manda o relatório que começamos.



pode me passar um resumo da sintaxe?

Claro. Aqui vai um resumo bem simples da sintaxe de Go para você usar no relatório.

Estrutura básica

Go

```
package main
```

```
import "fmt"
```

```
func main() {  
    fmt.Println("ola mundo")  
}
```

Variáveis

Com tipo:

Go

```
var nome string = "caio"  
var idade int = 21  
var altura float64 = 1.73  
var ativo bool = true
```

Forma mais curta:

Go

```
nome := "caio"  
idade := 21
```

O `:=` cria a variável e identifica o tipo automaticamente.

Entrada de dados

Go

```
var nome string  
  
fmt.Scan(&nome)
```

Com número:

Go

```
var idade int  
  
fmt.Scan(&idade)
```

O `&` é necessário no `Scan`.

Saída de dados

Go

```
fmt.Println("ola")
```

Com variável:

Go

```
nome := "caio"  
  
fmt.Println(nome)
```

Texto + variável:

Go

```
idade := 21

fmt.Println("idade:", idade)
```

Operações matemáticas

Go

```
a := 10
b := 5

soma := a + b
sub := a - b
mult := a * b
div := a / b
resto := a % b
```

If

Go

```
idade := 18

if idade >= 18 {
    fmt.Println("maior de idade")
}
```

If e else

Go

```
idade := 17

if idade >= 18 {
    fmt.Println("maior de idade")
} else {
    fmt.Println("menor de idade")
}
```

Else if

Go

```
nota := 70

if nota >= 90 {
    fmt.Println("otimo")
} else if nota >= 60 {
    fmt.Println("aprovado")
} else {
    fmt.Println("reprovado")
}
```

Comparações

Go

```
== igual
!= diferente
> maior
< menor
>= maior ou igual
<= menor ou igual
```

Operadores lógicos

Go

```
&& e
|| ou
! nao
```

Exemplo:

Go

```
idade := 20

if idade >= 18 && idade <= 60 {
    fmt.Println("idade valida")
}
```

For

Go não possui `while` tradicional. O `for` faz tudo.

For normal

Go

```
for i := 0; i < 5; i++ {  
    fmt.Println(i)  
}
```

Resultado:

```
0  
1  
2  
3  
4
```

Como while

Go

```
i := 0  
  
for i < 5 {  
    fmt.Println(i)  
    i++  
}
```

Loop infinito

Go

```
for {  
    fmt.Println("executando")  
}
```

Break

Interrompe o for .

Go

```
for i := 0; i < 10; i++ {  
    if i == 5 {  
        break  
    }  
  
    fmt.Println(i)  
}
```

Continue

Pula uma repetição.

Go

```
for i := 0; i < 5; i++ {  
    if i == 2 {  
        continue  
    }  
  
    fmt.Println(i)  
}
```

Arrays

Tamanho fixo:

Go

```
var numeros [5]int  
  
numeros[0] = 10  
numeros[1] = 20
```

Ou:

Go

```
numeros := [5]int{10, 20, 30, 40, 50}
```

Acessando:

Go

```
fmt.Println(numeros[0])
```

Slice

É parecido com array, mas mais flexível.

Go

```
numeros := []int{10, 20, 30}
```

Adicionar valor:

Go

```
numeros = append(numeros, 40)
```

Percorrer slice

Go

```
numeros := []int{10, 20, 30}

for i := 0; i < len(numeros); i++ {
    fmt.Println(numeros[i])
}
```

Ou com range :

Go

```
for _, numero := range numeros {
    fmt.Println(numero)
}
```

O _ significa que estamos ignorando o índice.

Também pode usar:

Go

```
for i, numero := range numeros {
    fmt.Println(i, numero)
}
```

Funções

Go

```
func soma(a int, b int) int {
    return a + b
}
```

Usando:

Go

```
resultado := soma(10, 5)

fmt.Println(resultado)
```

Também pode escrever:

Go

```
func soma(a, b int) int {  
    return a + b  
}
```

Função sem retorno

Go

```
func mensagem() {  
    fmt.Println("ola")  
}
```

Usando:

Go

```
mensagem()
```

Switch

Go

```
dia := 2  
  
switch dia {  
case 1:  
    fmt.Println("segunda")  
case 2:  
    fmt.Println("terca")  
case 3:  
    fmt.Println("quarta")  
default:  
    fmt.Println("outro dia")  
}
```

Strings

Go

```
nome := "caio"
```

Tamanho:

Go


```
fmt.Println(len(nome))
```

Concatenar:

Go

```
nome := "caio"
```

```
fmt.Println("ola " + nome)
```

Constantes

Go

```
const pi = 3.14
```

Conversão de tipos

Go

```
numero := 10
```

```
valor := float64(numero)
```

Exemplo:

Go

```
a := 10
```

```
b := 3
```

```
resultado := float64(a) / float64(b)
```

```
fmt.Println(resultado)
```

Sem converter, `10 / 3` usando `int` resulta em `3`.

Struct

É parecido com uma classe simples para guardar dados.

Go

```
type Pessoa struct {  
    nome string  
    idade int  
}
```

Criando:

Go

```

pessoa := Pessoa{
    nome: "caio",
    idade: 21,
}

fmt.Println(pessoa.nome)
```

Map

Funciona como chave e valor.

Go

```

idades := map[string]int{
    "caio": 21,
    "joao": 20,
}
```

Acessando:

Go

```

fmt.Println(idades["caio"])
```

Importar bibliotecas

Uma:

Go

```

import "fmt"
```

Mais de uma:

Go

```

import (
    "fmt"
    "math"
)
```

Math

Go

```
import "math"
```

Potência:

Go

```
resultado := math.Pow(2, 3)
```

Raiz quadrada:

Go

```
resultado := math.Sqrt(25)
```

Valor absoluto:

Go

```
resultado := math.Abs(-10)
```

Resumo do que mais provavelmente vai usar no relatório

Go

```
package main
```

```
import "fmt"
```

```
func main() {
```

```
    var numero int
```

```
    fmt.Scan(&numero)
```

```
    if numero > 0 {
```

```
        fmt.Println("positivo")
```

```
    } else {
```

```
        fmt.Println("negativo")
```

```
    }
```

```
}
```

E para repetição:

Go

```
package main
```

```
import "fmt"

func main() {
    for i := 1; i <= 10; i++ {
        fmt.Println(i)
    }
}
```

Para exercícios iniciais de Go, o principal é dominar `package main`, `import`, `func main()`, **variáveis**, `fmt.Scan`, `fmt.Println`, `if`, `else`, `for`, **funções**, **arrays/slices** e `range`.



Em Go:

= serve para atribuir um valor a uma variável que já existe.

Go

```
var idade int
```

```
idade = 21
```

Já := serve para criar a variável e atribuir um valor ao mesmo tempo:

Go

```
idade := 21
```

Nesse caso, o Go entende sozinho que `idade` é do tipo `int`.

Resumindo:

- `=` → variável já foi criada
- `:=` → cria a variável na hora

Exemplo:

Go

```
nome := "caio"
nome = "joao"
```

Aqui o primeiro `nome` é criado com `:=`, e depois o valor dele é alterado com `=`.

Um detalhe importante: `:=` normalmente é usado dentro de funções. Fora delas, você usa `var`.



Como faço um for infinito

Em Go, o `for` infinito é assim:

Go

```
for {
    fmt.Println("rodando")
}
```

Ele fica repetindo para sempre.

Para sair dele, você pode usar `break`:

Go

```
for {
    fmt.Println("rodando")

    break
}
```

Também é comum fazer assim:

Go

```
for {
    var numero int
    fmt.Scan(&numero)

    if numero == 0 {
        break
    }
}
```

Nesse exemplo, o `for` só termina quando o usuário digitar `0`.



da pra fazer switch sem expressao? como assim?

Sim. Em Go, dá para usar `switch` sem colocar uma expressão depois dele. Nesse caso, cada `case` vira uma condição booleana.

Exemplo:

Go

```
idade := 20

switch {
case idade < 18:
    fmt.Println("menor")
case idade >= 18 && idade < 60:
    fmt.Println("adulto")
default:
    fmt.Println("idoso")
}
```

É como se fosse um `if / else if / else`.

Esse:

Go

```
switch {
case nota >= 90:
    fmt.Println("a")
case nota >= 60:
    fmt.Println("aprovado")
default:
```



```
fmt.Println("reprovado")
}
```

funciona de forma parecida com:

Go

```
if nota >= 90 {
    fmt.Println("a")
} else if nota >= 60 {
    fmt.Println("aprovado")
} else {
    fmt.Println("reprovado")
}
```

Então, `switch` sem expressão é útil quando você quer testar várias condições diferentes.



Você gosta desta personalidade?



ⓘ ChatGPT pode cometer erros. Por isso, lembre-se de conferir informações relevantes.